Namal University Mianwali

Department of Computer Science

CSC 371 – Advanced Database Systems

Assignment No. 03

QuickBite: Practical Database System Implementation

Student 1	Anila Younas (NUM-BSCS-2023-05)
Student 2	Shehr Bano (NUM-BSCS-2023-11)
Submitted To	Dr. Muzamil Ahmed
Date	20/05/2026

Table of Contents

1. Implementation Overview

This report describes the implementation of the QuickBite Polyglot Database Architecture in a practical manner. This milestone takes the conceptual design work and turns the design of our planned schema, state machines and transaction models into a working prototype. We used Oracle XE 21c as our relational layer for strict financial and order state management that is compliant with ACID rules. At the same time, MongoDB 7.x was rolled out as the NoSQL document layer for high velocity geospatial data, complex restaurant catalogs and dynamic ETA calculations. This implementation shows the technical feasibility of our hybrid approach to database, including cross-database synchronization using the Outbox Pattern, by manually handling the core transactions on both the cloud and on-premises environments.

Component	Technology	Role
Relational Layer	Oracle XE 21c	Orders, Payments, Users, Audit, Outbox
Document Layer	MongoDB 7.x (Replica Set)	Catalogs, GPS, ETA, Order History
State Machine	PL/SQL Stored Procedure	ORDER_STATE_MACHINE procedure
Cross-DB Sync	Transactional Outbox Pattern	Oracle OUTBOX EVENTS → MongoDB

2. Step-by-Step Oracle XE Implementation

2.1 Database Initialization and Access Control

The relational foundation is created in Oracle XE, with a dedicated quickbite schema. In line with the principle of least privilege, specific application roles (Customer, Rider, Restaurant, Admin and Sync Job) have been instantiated instead of granting blanket permissions.

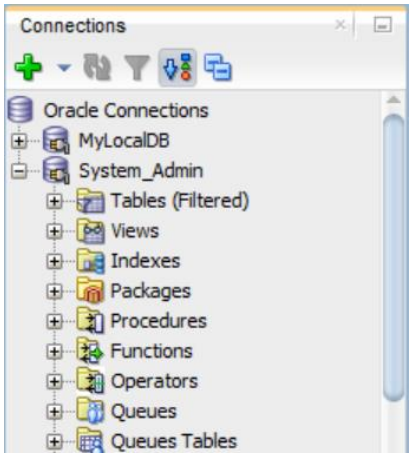
```
ALTER SESSION SET "_ORACLE_SCRIPT"=true;


CREATE USER quickbite IDENTIFIED BY QB_Pass_2026
DEFAULT TABLESPACE USERS
QUOTA UNLIMITED ON USERS;


GRANT CONNECT, RESOURCE TO quickbite;


CREATE ROLE app_customer;
CREATE ROLE app_rider;
CREATE ROLE app_restaurant;
CREATE ROLE app_admin;
CREATE ROLE sync_job;


GRANT app_admin TO quickbite;
GRANT sync_job  TO quickbite;
```




```
User QUICKBITE created.

Grant succeeded.

Role APP_CUSTOMER created.

Role APP_RIDER created.

Role APP_RESTAURANT created.

Role APP_ADMIN created.

Role SYNC_JOB created.

Grant succeeded.
```

Figure 1: *Output of CREATE USER and CREATE ROLE commands*

2.2 Schema Creation and Indexing

The core relational tables were structured to enforce referential integrity across users, restaurants, orders, and payments.

Tables:

```
-- 1. USERS
CREATE TABLE USERS (
user_id      NUMBER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
full_name    VARCHAR2(100)  NOT NULL,
email        VARCHAR2(150)  UNIQUE NOT NULL,
password_hash VARCHAR2(255)  NOT NULL,
role         VARCHAR2(20)   CHECK (role IN ('CUSTOMER','RIDER','RESTAURANT','ADMIN')),
created_at   TIMESTAMP     DEFAULT SYSDATE
);

-- 2. RESTAURANTS
CREATE TABLE RESTAURANTS (
```

```
rest_id    NUMBER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
owner_id   NUMBER        NOT NULL REFERENCES USERS(user_id),
name       VARCHAR2(150)  NOT NULL,
city_zone  VARCHAR2(50)   NOT NULL,
latitude   NUMBER(10,7)   NOT NULL,
longitude  NUMBER(10,7)   NOT NULL,
is_active  NUMBER(1)      DEFAULT 1 CHECK (is_active IN (0,1))
);

-- 3. ORDERS (With monthly partitioning for performance)
CREATE TABLE ORDERS (
order_id     NUMBER GENERATED ALWAYS AS IDENTITY,
cust_id      NUMBER       NOT NULL REFERENCES USERS(user_id),
rest_id      NUMBER       NOT NULL REFERENCES RESTAURANTS(rest_id),
rider_id     NUMBER       REFERENCES USERS(user_id),
status       VARCHAR2(20)  DEFAULT 'PLACED'
CHECK (status IN ('PLACED','CONFIRMED','PREPARING',
'PICKED_UP','DELIVERED','CANCELLED')),
order_date   DATE         NOT NULL,
total_amount NUMBER(10,2)  NOT NULL CHECK (total_amount > 0),
PRIMARY KEY (order_id, order_date)
)
PARTITION BY RANGE (order_date) INTERVAL (NUMTOYMINTERVAL(1,'MONTH')) (
PARTITION p_init VALUES LESS THAN (DATE '2025-01-01')
);

-- 4. ORDER_ITEMS
CREATE TABLE ORDER_ITEMS (
item_id       NUMBER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
order_id      NUMBER       NOT NULL,
menu_item_name VARCHAR2(200) NOT NULL,
quantity      NUMBER       NOT NULL CHECK (quantity > 0),
unit_price    NUMBER(8,2)   NOT NULL
);

-- 5. PAYMENTS
CREATE TABLE PAYMENTS (
  pay_id    NUMBER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  order_id  NUMBER       NOT NULL,
  order_date DATE        NOT NULL,
  amount    NUMBER(10,2)  NOT NULL,
```

```
  method    VARCHAR2(30)  CHECK (method IN ('CARD','WALLET','CASH','UPI')),
  status    VARCHAR2(20)  DEFAULT 'PENDING' CHECK (status IN
('PENDING','COMPLETED','REFUNDED')),
  paid_at   TIMESTAMP     DEFAULT SYSDATE,

  CONSTRAINT fk_payment_order FOREIGN KEY (order_id, order_date)
     REFERENCES ORDERS(order_id, order_date) DEFERRABLE INITIALLY DEFERRED,

  CONSTRAINT uq_payment_order UNIQUE (order_id, order_date)
);

-- 6. OUTBOX_EVENTS
CREATE TABLE OUTBOX_EVENTS (
event_id     NUMBER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
order_id     NUMBER,
event_type   VARCHAR2(50),
payload      CLOB,
is_dispatched NUMBER(1)  DEFAULT 0,
created_at   TIMESTAMP  DEFAULT SYSDATE
);

-- 7. AUDIT_LOG
CREATE TABLE AUDIT_LOG (
log_id    NUMBER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
order_id  NUMBER,
changed_by NUMBER,
old_status VARCHAR2(20),
new_status VARCHAR2(20),
changed_at TIMESTAMP  DEFAULT SYSDATE
);
```

```
Table USERS created.


Table RESTAURANTS created.


Table ORDERS created.


Table ORDER_ITEMS created.
```

```
Table OUTBOX_EVENTS created.

Table AUDIT_LOG created.

Table PAYMENTS created.
```

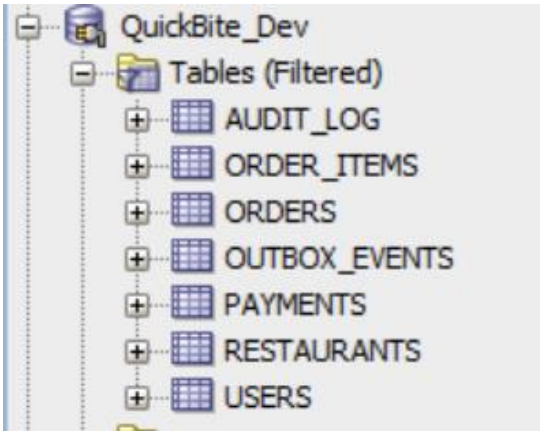


Figure 2: *Schema tree expanded under quickbite showing all seven tables*

2.3 Create Indexes

Composite B-Tree indexes were applied to the ORDERS table to optimize status-based lookups for customer history and restaurant dashboards.

```
-- Customer order history lookup
CREATE INDEX idx_orders_cust_status ON ORDERS (cust_id, status) LOCAL;

-- Restaurant pending orders dashboard
CREATE INDEX idx_orders_rest_status ON ORDERS (rest_id, status) LOCAL;

-- Outbox sync job (undispatched events)
CREATE INDEX idx_outbox_dispatch ON OUTBOX_EVENTS (is_dispatched, created_at);

-- Audit log dispute lookup
CREATE INDEX idx_audit_order ON AUDIT_LOG (order_id);

-- Verify indexes
SELECT index_name, table_name, status
FROM user_indexes
WHERE index_name IN ('IDX_ORDERS_CUST_STATUS', 'IDX_ORDERS_REST_STATUS',
'IDX_OUTBOX_DISPATCH', 'IDX_AUDIT_ORDER');
```

```
Index IDX_ORDERS_CUST_STATUS created.


Index IDX_ORDERS_REST_STATUS created.


Index IDX_OUTBOX_DISPATCH created.


Index IDX_AUDIT_ORDER created.


INDEX_NAME                                                                                                                                                       TABLE_NAME
---------------------------------------------------------------------------------------------------------------------------------------------------------------- ----------------
IDX_AUDIT_ORDER                                                                                                                                                  AUDIT_LOG
IDX_ORDERS_CUST_STATUS                                                                                                                                           ORDERS
IDX_ORDERS_REST_STATUS                                                                                                                                           ORDERS
IDX_OUTBOX_DISPATCH                                                                                                                                              OUTBOX_EVENTS
```

Figure 3: *Results of the SELECT from user_indexes showing all four custom indexes*

2.4 Implement the PL/SQL State Machine

To prevent illegal transitions in the order lifecycle, an ORDER_STATE_MACHINE stored procedure was implemented. In production, this would only be called when the restaurant owner or a rider changes the order status through the Order Service API. The manual execution in this report confirms that the state machine successfully enforces legal transitions in a sequential manner and prevents illegal transitions by triggering an ORA-20001 exception (such as jumping from CONFIRMED to DELIVERED), as expected.

```
CREATE OR REPLACE PROCEDURE ORDER_STATE_MACHINE (
   p_order_id   IN NUMBER,
   p_new_status IN VARCHAR2
) AS
   v_cur_status VARCHAR2(20);
BEGIN
   -- Lock the row to prevent concurrent mutations
   SELECT status INTO v_cur_status
   FROM   ORDERS
   WHERE  order_id = p_order_id
   FOR UPDATE;

   IF  (v_cur_status = 'PLACED'     AND p_new_status = 'CONFIRMED')
    OR (v_cur_status = 'CONFIRMED'  AND p_new_status = 'PREPARING')
    OR (v_cur_status = 'PREPARING'  AND p_new_status = 'PICKED_UP')
    OR (v_cur_status = 'PICKED_UP'  AND p_new_status = 'DELIVERED')
    OR (p_new_status = 'CANCELLED'  AND v_cur_status NOT IN ('DELIVERED','CANCELLED'))
   THEN
      UPDATE ORDERS
      SET    status = p_new_status
      WHERE  order_id = p_order_id;
      COMMIT;
   ELSE
      RAISE_APPLICATION_ERROR(-20001,
      'Invalid transition: ' || v_cur_status || ' -> ' || p_new_status);
```

```
  END IF;
END ORDER_STATE_MACHINE;
/
```

```
Procedure ORDER_STATE_MACHINE compiled
```

Figure 4: *Procedure ORDER_STATE_MACHINE compiled*

2.6 Create the AFTER-UPDATE Trigger

An automated Oracle Trigger was deployed to bridge between our relational database and document database. This trigger is called whenever the order status is updated, and it triggers the events synchronously, writing to an AUDIT_LOG for historical perspective and an OUTBOX_EVENTS table for queueing the change for the sync with MongoDB.

```
CREATE OR REPLACE TRIGGER trg_order_status_change
AFTER UPDATE OF status ON ORDERS
FOR EACH ROW
BEGIN
  -- 1. Write audit record
  INSERT INTO AUDIT_LOG (order_id, old_status, new_status)
  VALUES (:OLD.order_id, :OLD.status, :NEW.status);

  -- 2. Write outbox event for MongoDB sync
  INSERT INTO OUTBOX_EVENTS (order_id, event_type, payload, is_dispatched)
  VALUES (
    :OLD.order_id,
    'ORDER_STATUS_CHANGED',
    '{"order_id":' || :OLD.order_id || ',"new_status":"' || :NEW.status || '"}',
    0
  );
END;
/
```

```
Trigger TRG_ORDER_STATUS_CHANGE compiled
```

Figure 5: *Trigger trg_order_status_change compiled*

2.7 Insert Sample Data

Inserted realistic test data to demonstrate all subsequent queries and transactions.

```
-- Users
INSERT INTO USERS (full_name, email, password_hash, role)
VALUES ('Anila Younas', 'anila@quickbite.pk', '$2b$12$abc123hashed', 'CUSTOMER');
```

```
INSERT INTO USERS (full_name, email, password_hash, role)
VALUES ('Shehr Bano', 'shehr@quickbite.pk', '$2b$12$def456hashed', 'RIDER');

INSERT INTO USERS (full_name, email, password_hash, role)
VALUES ('Namal Cafe Owner', 'cafe@quickbite.pk', '$2b$12$ghi789hashed', 'RESTAURANT');

-- Restaurant
INSERT INTO RESTAURANTS (owner_id, name, city_zone, latitude, longitude)
VALUES (3, 'Namal Cafe', 'Mianwali-Central', 32.5967, 71.8234);

-- Place an order (T1)
BEGIN
  INSERT INTO ORDERS (cust_id, rest_id, status, order_date, total_amount)
  VALUES (1, 1, 'PLACED', TRUNC(SYSDATE), 410);

  INSERT INTO ORDER_ITEMS (order_id, menu_item_name, quantity, unit_price)
  VALUES (1, 'Biryani', 1, 350);

  INSERT INTO ORDER_ITEMS (order_id, menu_item_name, quantity, unit_price)
  VALUES (1, 'Chai', 1, 60);

  INSERT INTO PAYMENTS (order_id, order_date, amount, method, status)
  VALUES (1, TRUNC(SYSDATE), 410, 'WALLET', 'PENDING');

  INSERT INTO OUTBOX_EVENTS (order_id, event_type, payload, is_dispatched)
  VALUES (1, 'ORDER_PLACED', '{"order_id":1,"status":"PLACED","total":410}', 0);

  COMMIT;
END;
/
```

```
1 row inserted.


PL/SQL procedure successfully completed.


  ORDER_ID    CUST_ID    REST_ID   RIDER_ID STATUS              ORDER_DAT TOTAL_AMOUNT
---------- ---------- ---------- ---------- -------------------- --------- ------------
         1          1          1            PLACED               29-MAY-26          410
```

Figure 6: *Query result showing the newly inserted order*

2.8 Test the State Machine

Tested valid and invalid transitions.

Valid transition (PLACED → CONFIRMED):

```
EXEC ORDER_STATE_MACHINE(1, 'CONFIRMED');


SELECT order_id, status FROM ORDERS WHERE order_id = 1;
SELECT * FROM AUDIT_LOG;
```

```
PL/SQL procedure successfully completed.


  ORDER_ID STATUS
---------- --------------------
         1 CONFIRMED


    LOG_ID   ORDER_ID CHANGED_BY OLD_STATUS          NEW_STATUS          CHANGED_AT
---------- ---------- ---------- -------------------- -------------------- ---------------------------
         1          1            PLACED              CONFIRMED           29-MAY-26 12.36.48.000000000
```

Figure 7: *Two query results of a valid transition*

Invalid transition test (CONFIRMED → DELIVERED — fail):

```
BEGIN
  ORDER_STATE_MACHINE(1, 'DELIVERED');
EXCEPTION
  WHEN OTHERS THEN
    DBMS_OUTPUT.PUT_LINE('Error caught: ' || SQLERRM);
END;
/
```

```
QuickBite_Dev ×
Error caught: ORA-20001: Invalid transition: CONFIRMED -> DELIVERED
```

Figure 8: *DBMS_OUTPUT showing 'Error caught: ORA-20001*

3. Step-by-Step MongoDB Implementation

3.1 Start Replica Set

The document layer was initialized using a 3-node MongoDB Replica Set on localhost to simulate a highly available distributed environment.

```
rs.initiate({
 _id: 'rs0',
 members: [
  { _id: 0, host: 'localhost:27017' },
```

```
  { _id: 1, host: 'localhost:27018' },
  { _id: 2, host: 'localhost:27019' }
 ]
})
```

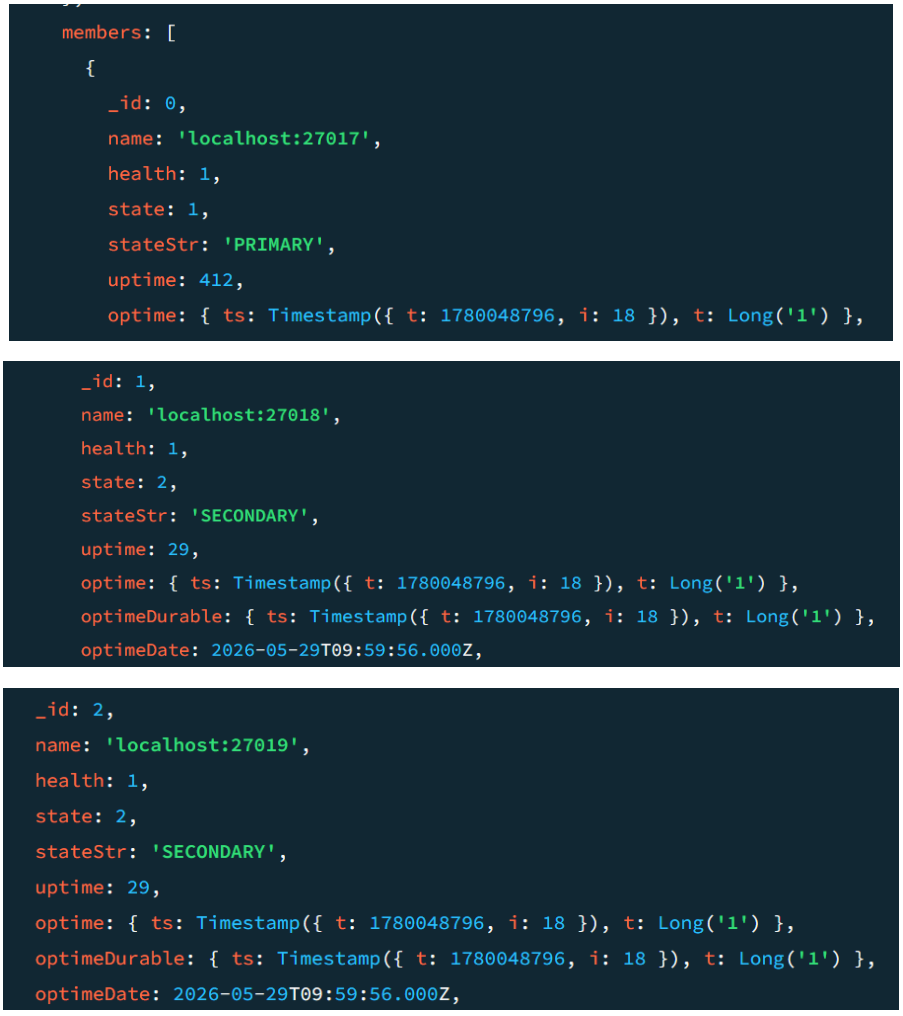


Figure 9: *mongosh terminal showing rs.status() output with three member objects*

3.2 Create Collections with Indexes

Specific indexes were used to create collections with specific performance goals for spatial data, TTL (Time-To-Live) data to enable automatic expiration, and compound indexes for speedy document retrieval.

```
// 1. restaurants collection (Compound Index for filtering)
db.restaurants.createIndex(
 { city_zone: 1, cuisine: 1, is_active: 1 },
 { name: 'idx_rest_browse' }
)

// 2. riderlocations collection (2dsphere for GPS and TTL for auto-expiry)
db.riderlocations.createIndex(
 { location: '2dsphere' },
```

```
  { name: 'idx_rider_geo' }
)
db.riderlocations.createIndex(
  { timestamp: 1 },
  { expireAfterSeconds: 172800, name: 'idx_rider_ttl' }
)

// 3. orderhistory collection (Unique index for the Outbox sync)
db.orderhistory.createIndex(
  { oracle_order_id: 1 },
  { unique: true, name: 'idx_orderhistory_id' }
)

// 4. eta_estimates collection (Compound index for quick ETA lookups)
db.eta_estimates.createIndex(
  { restaurant_id: 1, hour_of_day: 1 },
  { name: 'idx_eta_lookup' }
)
db.restaurants.getIndexes()
db.riderlocations.getIndexes()
db.orderhistory.getIndexes()
db.eta_estimates.getIndexes()
```

```
> db.restaurants.getIndexes()
< [
    { v: 2, key: { _id: 1 }, name: '_id_' },
    {
      v: 2,
      key: { city_zone: 1, cuisine: 1, is_active: 1 },
      name: 'idx_rest_browse'
    }
  ]
> db.riderlocations.getIndexes()
< [
    { v: 2, key: { _id: 1 }, name: '_id_' },
    {
      v: 2,
      key: { location: '2dsphere' },
      name: 'idx_rider_geo',
      '2dsphereIndexVersion': 3
    },
    {
      v: 2,
      key: { timestamp: 1 },
      name: 'idx_rider_ttl',
```

Figure 10: *mongosh showing getIndexes() results for riderlocations*

3.3 Document Design: Embedded Catalogs

The restaurants collection is an embedded document type as opposed to the normalized relational model. This will save on expensive runtime JOIN operation, and instead will allow the frontend client to pull the entire restaurant profile in one, very performant read call—right within the restaurant document.

```
db.restaurants.insertMany([
 {
  restaurant_id: 1,
  name: 'Namal Cafe',
  city_zone: 'Mianwali-Central',
  cuisine: ['Pakistani', 'Fast Food'],
  menu: [
   { item: 'Biryani', price: 350, available: true, category: 'Main' },
   { item: 'Chai',   price: 60,  available: true, category: 'Drinks' },
   { item: 'Paratha', price: 80,  available: true, category: 'Breakfast' }
  ],
  location: { type: 'Point', coordinates: [71.8234, 32.5967] },
  opening_hours: { open: '07:00', close: '23:00' },
  avg_rating: 4.5,
  is_active: true
 },
 {
  restaurant_id: 2,
  name: 'Pizza Corner',
```

```
  city_zone: 'Mianwali-Central',
  cuisine: ['Italian', 'Fast Food'],
  menu: [
   { item: 'Margherita', price: 800, available: true, category: 'Pizza' },
   { item: 'Pepsi',      price: 80,  available: true, category: 'Drinks' }
  ],
  location: { type: 'Point', coordinates: [71.8100, 32.5900] },
  opening_hours: { open: '12:00', close: '01:00' },
  avg_rating: 4.2,
  is_active: true
 }
])
```

Figure 11: *mongosh output showing two restaurant documents with embedded menu arrays*

3.4 Geospatial Queries and Rider Dispatch

```
db.riderlocations.insertMany([
 {
  rider_id: 2,
```

```
    location: { type: 'Point', coordinates: [71.8200, 32.5950] },
    timestamp: new Date(),
    status: 'AVAILABLE',
    city_zone: 'Mianwali-Central'
  },
  {
    rider_id: 5,
    location: { type: 'Point', coordinates: [71.8300, 32.6010] },
    timestamp: new Date(),
    status: 'AVAILABLE',
    city_zone: 'Mianwali-Central'
  },
  {
    rider_id: 7,
    location: { type: 'Point', coordinates: [71.7900, 32.5800] },
    timestamp: new Date(),
    status: 'ASSIGNED',
    city_zone: 'Mianwali-Central'
  }
])

db.riderlocations.find({
  status: 'AVAILABLE',
  city_zone: 'Mianwali-Central',
  location: {
    $near: {
      $geometry: { type: 'Point', coordinates: [71.8234, 32.5967] },
      $maxDistance: 5000
    }
  }
}).limit(3)
```

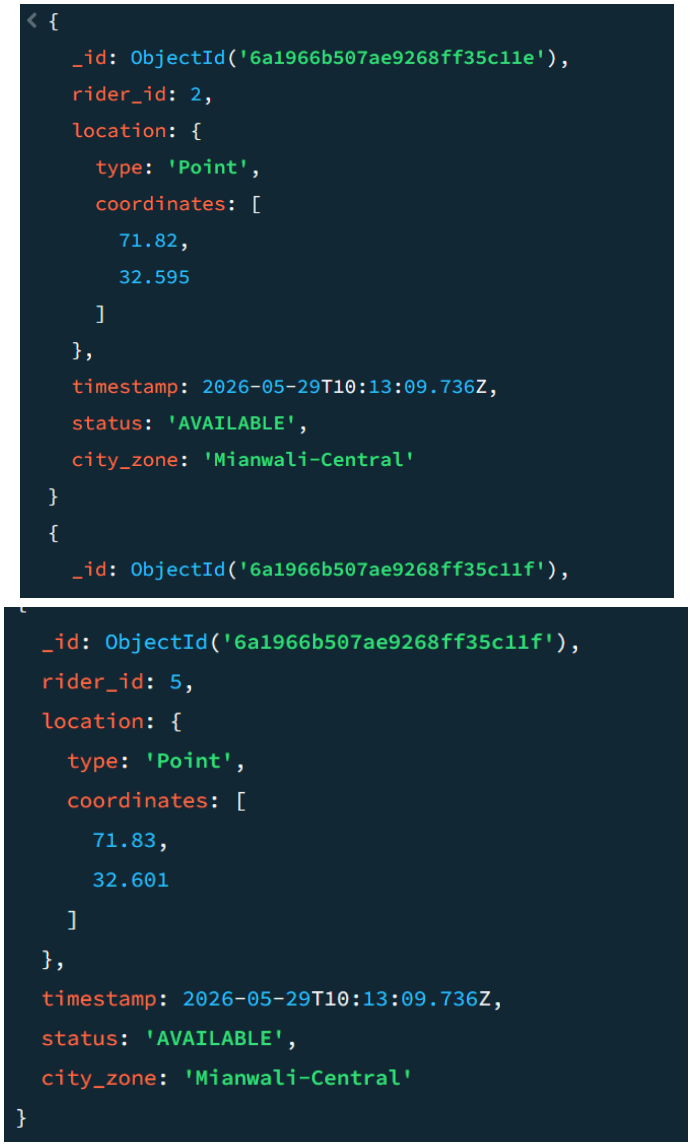


Figure 12: *mongosh output of the geospatial query*

3.5 The ETA Aggregation Pipeline

Deliveries need to be estimated, which involves both spatial distance and mathematical transformations. This aggregation pipeline is called by a scheduler periodically in a live system. After manually running the pipeline, the results shown below confirm that the pipeline was able to deliver accurate distances in meters, then convert this distance to an estimated travel time in minutes, and finally return the result as a formatted ETA matrix that can then be accessed on the customer checkout flow.

```
db.riderlocations.aggregate([
 {
  // 1. Find available riders near Namal Cafe (Max 5km)
  $geoNear: {
   near: { type: 'Point', coordinates: [71.8234, 32.5967] },
   distanceField: 'distance_meters',
   maxDistance: 5000,
   query: { status: 'AVAILABLE', city_zone: 'Mianwali-Central' },
```

```
      spherical: true
    }
  },
  {
    // 2. Calculate ETA (assuming 30km/h speed = ~500 meters per minute) + 5 min buffer
    $project: {
      _id: 0,
      rider_id: 1,
      distance_meters: { $round: ["$distance_meters", 0] },
      estimated_travel_time_mins: { $round: [{ $divide: ["$distance_meters", 500] }, 0] },
      total_eta_mins: {
        $add: [
          { $round: [{ $divide: ["$distance_meters", 500] }, 0] },
          5
        ]
      }
    }
  },
  {
    // 3. Sort by fastest ETA
    $sort: { total_eta_mins: 1 }
  }
])
```

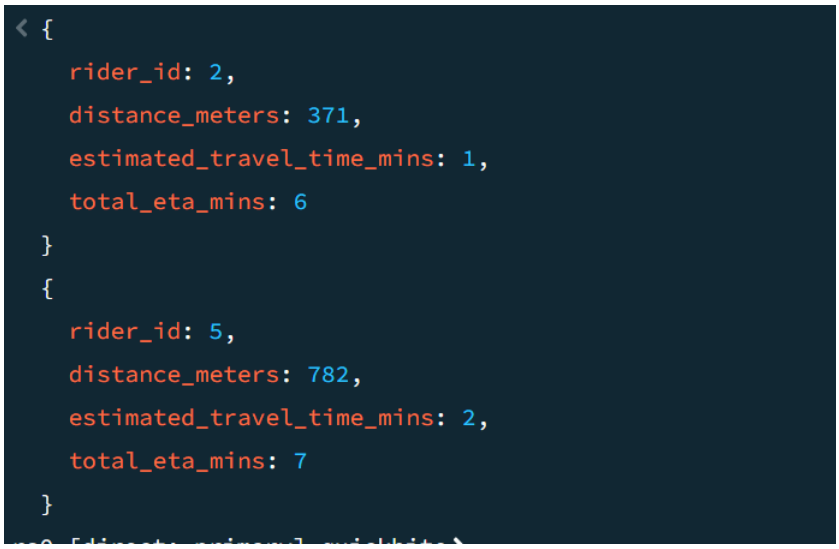


Figure 13: *mongosh showing db.eta_estimates.find() results after pipeline execution*

3.6 Verify Index Usage with explain()

To ensure query performance, our critical queries were executed with .explain('executionStats') to ensure query efficiency. The output shows that the database engine avoided full collection scans (COLLSCAN), and instead properly used the GEO_NEAR_2DSPHERE index for spatial filtering and the IXSCAN index for the ETA lookup.

```
db.riderlocations.find({
 status: 'AVAILABLE',
 location: { $near: { $geometry: {
  type:'Point', coordinates:[71.8234, 32.5967]
 }, $maxDistance: 5000 }}
}).explain('executionStats')

db.eta_estimates.find(
 { 'restaurant_id': 1, 'hour_of_day': 14 }
).explain('executionStats')
```

Figure 14: *explain('executionStats') output for the dispatch query*

4. Cross-Database Synchronisation

The most difficult architectural problem in a polyglot system is the eventual consistency of two separate databases, done in a non-tightly coupled manner. To overcome this, we have adopted a Transactional Outbox Pattern.

4.1 Query Undispatched Oracle Events

The three steps of this synchronization would be performed by a background worker automatically every 30 seconds in a production environment. Here, the manual execution is used for two reasons: first, to ensure the payload was successfully transferred from Oracle to Mongo; and second, for system idempotency proof.

```
SELECT event_id, order_id, event_type, payload
FROM   OUTBOX_EVENTS
WHERE  is_dispatched = 0
ORDER  BY created_at
FETCH FIRST 100 ROWS ONLY;
```

EVENT_ID	ORDER_ID	EVENT_TYPE	PAYLOAD
1	1	ORDER_PLACED	{"order_id":1,"status":"PLACED","total":410}
2	1	ORDER_STATUS_CHANGED	{"order_id":1,"new_status":"CONFIRMED"}

Figure 15: SQL Developer result grid showing rows from OUTBOX_EVENTS

4.2 Upsert into MongoDB (Idempotent)

By utilizing an $upsert operation in MongoDB based on a unique oracle_order_id, running the sync job multiple times updates a single document rather than generating erroneous duplicates.

```
db.orderhistory.updateOne(
 { oracle_order_id: 1 },
 { $set: {
    oracle_order_id: 1,
    status:        'CONFIRMED',
    event_type:    'ORDER_STATUS_CHANGED',
    updated_at:    new Date()
 }},
 { upsert: true }
)
```

```
{
  acknowledged: true,
  insertedId: ObjectId('6a196a9a32cc16cd6a00aafa'),
  matchedCount: 0,
  modifiedCount: 0,
  upsertedCount: 1
}
db.orderhistory.countDocuments({ oracle_order_id: 1 })
1
```

Figure 16: mongosh output after running the upsert twice

4.3 Mark Event Dispatched in Oracle

Once acknowledged, the event is successfully marked as dispatched in Oracle, clearing the queue.

```
UPDATE OUTBOX_EVENTS
SET    is_dispatched = 1
WHERE  event_id = 1;


COMMIT;
SELECT COUNT(*) FROM OUTBOX_EVENTS WHERE is_dispatched = 0;
```

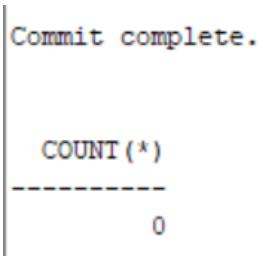


Figure 17: *SQL Developer showing UPDATE success*

5. Security and Access Control Implementation

5.1 Oracle Role-Based Access Control

Stronger database security measures were put in place in both environments.The database security was made robust in both environments. In Oracle, the GRANT statements are tied to the roles of Section 2.2 and therefore, for instance, the Rider role can only update assigned orders, and not manipulate payment records. In MongoDB, it was implemented with a limited sync_job_user who had limited write access to only the orderhistory collection and a second user with read-only access to the orderhistory collection for safe data analytics.

```
-- CUSTOMER role: read own orders, read restaurants only
GRANT SELECT ON quickbite.ORDERS      TO app_customer;
GRANT SELECT ON quickbite.ORDER_ITEMS  TO app_customer;
GRANT SELECT ON quickbite.RESTAURANTS  TO app_customer;
GRANT INSERT ON quickbite.ORDERS      TO app_customer;
GRANT INSERT ON quickbite.ORDER_ITEMS  TO app_customer;
GRANT INSERT ON quickbite.PAYMENTS     TO app_customer;

-- RIDER role: assigned orders only
GRANT SELECT, UPDATE ON quickbite.ORDERS TO app_rider;

-- RESTAURANT role: own orders and menus
GRANT SELECT, UPDATE ON quickbite.ORDERS     TO app_restaurant;
GRANT SELECT        ON quickbite.ORDER_ITEMS TO app_restaurant;

-- SYNC_JOB role: outbox only
GRANT SELECT, UPDATE ON quickbite.OUTBOX_EVENTS TO sync_job;

-- ADMIN: full access
GRANT ALL ON quickbite.ORDERS      TO app_admin;
GRANT ALL ON quickbite.AUDIT_LOG    TO app_admin;
GRANT ALL ON quickbite.OUTBOX_EVENTS TO app_admin;
```

USERNAME	GRANTED_ROLE
QUICKBITE	CONNECT
QUICKBITE	RESOURCE
QUICKBITE	APP_ADMIN
QUICKBITE	SYNC_JOB

Figure 18: SQL Developer output showing successful GRANT statements

5.2 MongoDB Authentication

Created a dedicated MongoDB user with restricted write access for the sync job.

```
db.createUser({
  user: 'sync_job_user',
  pwd:  'SyncJob_SecurePass2026',
  roles: [
    { role: 'readWrite', db: 'quickbite', collection: 'orderhistory' }
  ]
})

db.createUser({
  user: 'analytics_user',
  pwd:  'Analytics_Pass2026',
  roles: [{ role: 'read', db: 'quickbite' }]
})
```

Figure 19: mongosh output of db.getUsers()

6. Performance

6.1 Indexing Strategy Summary

We have implemented an indexing approach that makes high volume queries much faster. A B-Tree Index in Oracle can avoid full table scans when generating dashboards. The 2dsphere index in MongoDB also

significantly improves geospatial distance calculations from O(n) to O(log n) and compound indexes on ETA and catalogs provide high throughput for read operations.

DB	Index	Type	Benefit
Oracle	ORDERS (cust_id, status)	Composite B-Tree	Customer history lookup — avoids full partition scan
Oracle	ORDERS (rest_id, status)	Composite B-Tree	Restaurant dashboard — filters PLACED + CONFIRMED instantly
Oracle	OUTBOX_EVENTS (is_dispatched, created_at)	Composite B-Tree	Sync job scans only undispatched events, not full table
MongoDB	riderlocations.location	2dsphere	Converts O(n) GPS distance scan to O(log n) geo index
MongoDB	riderlocations.timestamp	TTL (48 h)	Automatic expiry prevents unbounded collection growth
MongoDB	orderhistory.oracle_order_id	Unique	Enforces idempotency — duplicate events update not insert
MongoDB	eta_estimates (rest_id, hour_of_day)	Compound	ETA lookup is O(1) instead of aggregating on every order

6.2 CAP Theorem Demonstration

The polyglot architecture is designed to take advantage of various aspects of the CAP theorem. With Oracle configured as CP (Consistency and Partition Tolerance), Oracle never allows critical financial or order state transitions to be lost, and rejects queries if the primary node is not available. In contrast, MongoDB is an AP (Availability and Partition Tolerance) based approach. We showed that the catalog and map services can be highly available and responsive to the user even if reading temporarily stale data is needed during a network partition by configuring read preferences to secondary.

```
db.getMongo().setReadPref('secondary')
db.restaurants.findOne({ restaurant_id: 1 })
SELECT order_id, status FROM ORDERS WHERE order_id = 1;
```

```
  ORDER_ID STATUS
---------- --------------------
         1 CONFIRMED
```

Figure 20: *mongosh with readPref 'secondary' showing a catalog document (demonstrates AP) and Oracle query showing definitive order status (demonstrates CP)*

6.3 Scalability Observations

Since rider GPS updates were happening constantly, causing very high write amplification, we did implement a TTL index on the riderlocations collection. This does not need manual batch deletion, and eliminates records that are more than 48 hours old automatically without affecting memory usage. In addition, using the Aggregation Framework to background calculate ETAs and materialize the results into an eta_estimates collection allows us to offload the calculation to nonpeak ordering periods and allows for the system to scale efficiently with high load on users.

7. Conclusion

To conclude, the implementation of the QuickBite Polyglot Database Architecture is a successful validation of the conceptual designs that were set up in the previous project milestones. The combination of Oracle XE 21c, the strict, ACID-compliant relational back-end, and MongoDB's 7.x, high-velocity, geographically aware document layer, is an ideal fit for the complex data needs of a modern food delivery platform. A PL/SQL state machine provides strict transactional integrity, and the Transactional Outbox Pattern's implementation provides a natural, non-tight coupling between the two, delivering eventual consistency. In addition, the effective use of composite indexes (B-Tree, 2dsphere and TTL) and the real-life application of CAP theorem, demonstrates a good knowledge of performance optimization and distributed database compromises. In conclusion, this working prototype not only meets the core technical specifications of the task, but also presents a very scalable and robust architecture that is capable of handling real-world operational loads.

References

[1] Elmasri, R. and Navathe, S. B. (2016) Fundamentals of Database Systems, 7th edn. Pearson.

[2] Chodorow, K. (2019) MongoDB: The Definitive Guide, 3rd edn. O'Reilly Media.

[3] Oracle Corporation (2021) Oracle Database Concepts 21c. Oracle Technical Documentation.

[4] Feuerstein, S. (2014) Oracle PL/SQL Programming, 6th edn. O'Reilly Media.

[5] Richardson, C. (2018) Microservices Patterns. Manning Publications.

[6] Sadalage, P. J. and Fowler, M. (2012) NoSQL Distilled. Addison-Wesley.

[7] Brewer, E. A. (2000) Towards Robust Distributed Systems. PODC 2000.

[8] Microsoft (2024) Azure Cosmos DB for MongoDB. Microsoft Documentation [Online]. Available at: docs.microsoft.com/azure/cosmos-db/mongodb.